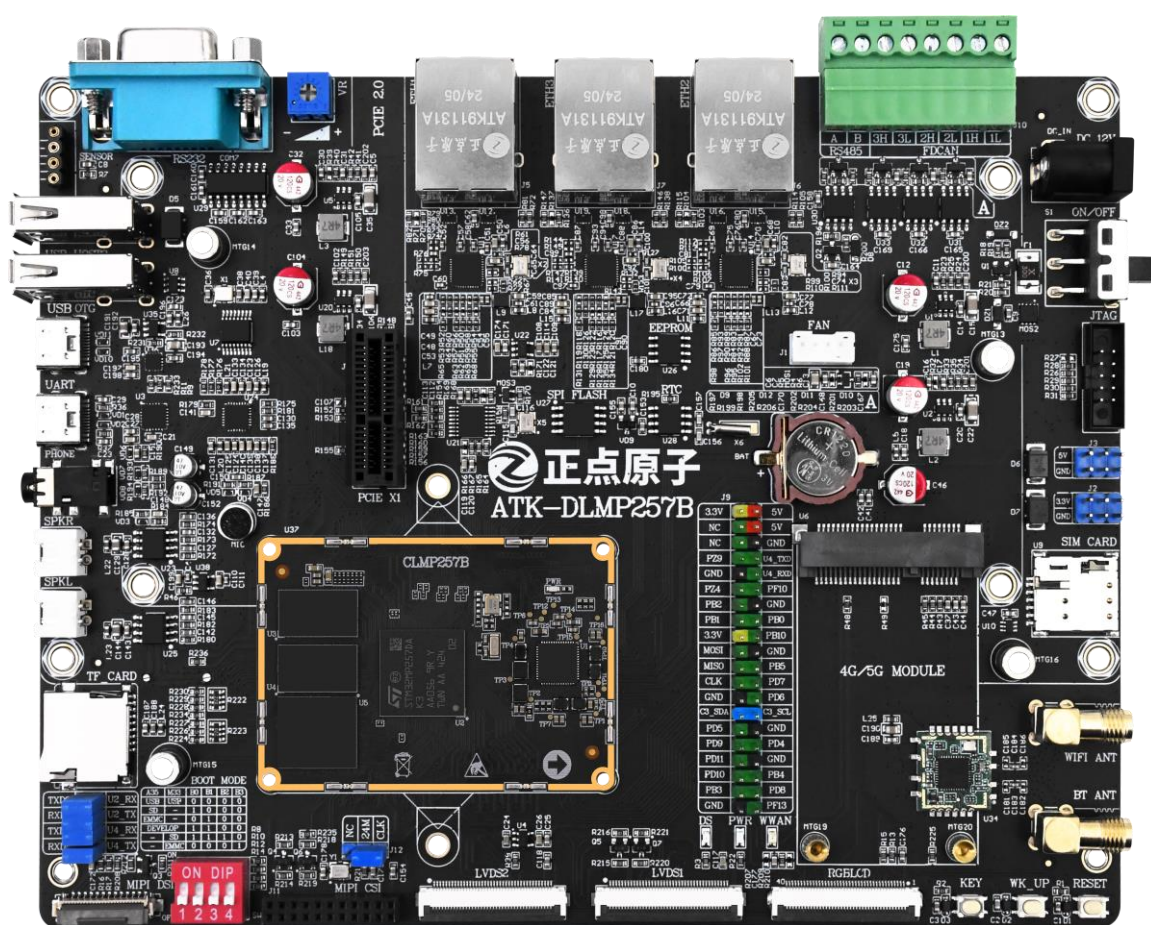


ATK-DLMP257B

出厂系统外设复用手册

V1.2





正点原子公司名称 : 广州市星翼电子科技有限公司
原子哥在线教学平台 : www.yuanzige.com
开源电子网 / 论坛 : <http://www.openedv.com/forum.php>
正点原子淘宝店铺 : <https://openedv.taobao.com>
正点原子官方网站 : www.alientek.com
正点原子 B 站视频 : <https://space.bilibili.com/394620890>

电话: 020-38271790 传真: 020-36773971

请关注正点原子公众号, 资料发布更新我们会通知。



扫码关注正点原子公众号

免责声明

本文档所提及的产品规格和使用说明仅供参考，如有内容更新，恕不另行通知；除非有特殊约定，本文档仅作为产品指导，所作陈述均不构成任何形式的担保。本文档版权归广州市星翼电子科技有限公司所有，未经公司的书面许可，任何单位和个人不得以营利为目的进行任何方式的传播。

为了得到最新版本的产品信息，请用户定时访问正点原子资料下载中心或者与淘宝正点原子旗舰店客服联系索取。感谢您的包容与支持。

文档更新说明

版本	版本更新说明	负责人	校审	发布日期
V1.0	正式发布	正点原子	正点原子	2025.4.1
V1.1	将手册中所有的 stm32mp25-pinctrl-atk.dtsi 描述修改成 stm32mp25-pinctrl-atk-ddr-2GB.dtsi 和 stm32mp25-pinctrl-atk-ddr-1GB.dtsi	正点原子	正点原子	2025.4.25
V1.2	补充第四章、第六章内容	正点原子	正点原子	2025.8.4

目录

前言	1
第一章 出厂源码获取与编译	2
第二章 多路 GPIO 复用	3
第三章 多路 UART/USART 复用	4
3.1 确认引脚所属 UART	4
3.2 配置设备树中的 UART 主节点	4
3.3 配置 UART 节点	5
3.4 修改引脚复用配置	6
3.5 UART 功能调试	7
第四章 9 路 UART 复用案例	8
4.1 屏蔽串口引脚资源冲突的设备树	8
4.2 修改系统支持串口实例数量	11
4.3 USART1 (/dev/ttySTM1)	12
4.4 USART2 (/dev/ttySTM0)	12
4.5 USART3 (/dev/ttySTM2)	12
4.6 UART4 (/dev/ttySTM3)	14
4.1 UART5 (/dev/ttySTM4)	15
4.1 USART6 (/dev/ttySTM5)	17
4.1 UART7 (/dev/ttySTM6)	18
4.1 UART8 (/dev/ttySTM7)	19
4.1 UART9 (/dev/ttySTM8)	21
4.1 LPUART1 (/dev/ttySTM9)	22
第五章 多路 ADC 复用	24
5.1 确认引脚所属 ADC 控制器	24
5.2 配置设备树中的 ADC 主节点	24
5.3 配置 ADC 通道	25
5.4 修改引脚复用配置	26
5.5 更新设备树以支持多个 ADC 通道	26
5.6 ADC 功能测试	28
第六章 ENET 裁剪	30
6.1 单独使用 ETH1 (网口 ENET1)	30
6.2 单独使用 ETH2 (网口 ENET2)	35

前言

本章介绍了在 STM32MP257 平台上配置和复用多种外设的方法，旨在帮助开发者充分利用处理器的多功能引脚，提高系统的扩展能力和硬件复用率。

首先，通过获取和编译出厂源码，建立完整的开发环境，确保后续配置能够顺利进行。然后，详细介绍了如何在设备树中配置和复用外设，包括引脚的功能定义、复用设置以及设备树节点配置方法。通过本章的学习，开发者可以掌握在 STM32MP257 平台上配置和复用常见外设的完整方法，为嵌入式系统的功能扩展和硬件优化提供坚实的基础。

第一章 出厂源码获取与编译

执行本文档所有操作前, 请先按照“\STM32MP257 开发板 A 盘-基础资料\10、用户手册\《【正点原子】ATK-DLMP257B 出厂系统源码使用指南》”文档内容, 获取出厂源码, 确保当前虚拟机环境能够成功编译出厂源码。

第二章 多路 GPIO 复用

对于 STM32MP257 核心板，大部分管脚均支持复用为 GPIO 功能。在 STM32MP2 中，当某个 IO 管脚用于 GPIO 功能时，无需在设备树中创建对应的 pinctrl 节点。

为实现管脚的 GPIO 功能复用，需首先确认该管脚在内核或 U-Boot 的设备树中是否被配置为其他外设功能。如果已被占用，需禁用相关设备树节点或注释掉对应的管脚复用代码。管脚释放后，即可通过系统的 GPIO 指令对其进行高低电平的设置。完成设置后，可结合万用表测试电平变化，以确认管脚是否正常工作。

详细操作教程请参考开发板资料路径：“开发板光盘 A-基础资料\09、文档教程(非常重要)\《【正点原子】ATK-DLMP257 嵌入式 Linux C 应用编程指南》文档的《第十六章 GPIO 应用编程》”。

第三章 多路 UART/USART 复用

3.1 确认引脚所属 UART

STM32MP257 支持多达 9 路 UART/USART, 还有一路 LPUART。在配置多路 UART 时, 必须特别注意设备树中定义的引脚复用是否与其他外设 (如 I2C、SPI 等) 发生冲突。如果需要为不同的 UART/USART 配置独立的引脚, 大家应检查目标芯片的引脚复用映射表, 以确认是否支持所需的配置。为了确保目标引脚可以复用为 UART/USART 功能, 建议参考《【正点原子】ATK-CLMP257B 核心板接口数据手册》进行详细查询。

引脚序号	引脚号	核心板引脚名称	出厂系统默认配置/开发板接口	GPIO编号	功能描述	引脚类型	电平电压	AF0	AF1	AF2
1	GPIO				接焊					
2	GPIO				接焊					
3	GPIO				接焊					
4	GPIO				接焊					
5	GPIO				接焊					
6										
7	VBUS_SSD07	TF Card			核心板电源输出接口	电源	3.3V/1.8V			
8										
9	VBUS_1V8	CORE BOARD			核心板电源输出接口	电源	1.8V			
10										
11	VBUS_OUT	CORE BOARD			核心板电源输出接口	电源	1.8V			
12										
13	VBAT	CORE BOARD			核心板电池输入引脚	电源	3.3V			
14										
15	KEY0			PA5	按键信号	输入/输出	3.3V	--	--	--
16	GPIO_PWR_CTL	USB OTG		PA5	USB OTG 电源控制信号	输入/输出	3.3V	--	NC01	LPTIM1_RTE
17	USB_LPM			PA5	USB OTG 中断信号	输入/输出	3.3V	SBTBS	SBTBS0	--
18	RTC_RST	RTC		PA7	RTC 复位信号	输入/输出	3.3V	--	--	--
19	VBUS_RQ	TEMPERATURE SENSOR		PA7	温度传感器行通信信号	输入/输出	3.3V	--	--	LPTIM1_RTE
20										
21	GPIO				接焊					
22	ETH0_ETH125			PA8	千兆以太网 0 外部时钟信号	输入/输出	1.8V	--	RTC_RSTIN	--
23	ETH0_ETH0			PA8	千兆以太网 0 数据信号	输入/输出	3.3V	--	--	SPDTRM1_I2C
24	ETH0_ETH1			PA8	千兆以太网 1 数据信号	输入/输出	3.3V	--	RTC_RSTIN	SPDTRM1_I2C
25	ETH0_ETH125			PA8	千兆以太网 0 外部时钟信号	输入/输出	3.3V	--	--	SPDTRM1_I2C
26	GPIO				接焊					
27	ETH0_ETH0			PA7	千兆以太网 0 数据信号	输入/输出	1.8V	--	--	--
28	ETH0_ETH1			PA8	千兆以太网 1 数据信号	输入/输出	1.8V	--	LPTIM1_RTE	--
29	ETH0_ETH2			PA9	千兆以太网 2 数据信号	输入/输出	1.8V	--	NC01	--
30	ETH0_ETH3			PA10	千兆以太网 3 数据信号	输入/输出	1.8V	--	--	SPDTRM1_I2C
31	ETH0_ETH_CTL			PA11	千兆以太网 控制信号	输入/输出	1.8V	--	--	SPDTRM1_I2C
32	ETH0_ETH_CTL			PA11	千兆以太网 控制信号	输入/输出	1.8V	--	--	SPDTRM1_I2C
33	GPIO				接焊					
34	ETH0_ETH0			PA11	千兆以太网 0 数据信号	输入/输出	1.8V	--	--	SPDTRM1_I2C

图 3.1.1 ATK-CLMP257B 核心板接口数据手册

3.2 配置设备树中的 UART 主节点

设备树文件位于 stm32mp251.dtsi, 需要在 stm32mp257d-atk-ddr-1GB.dts 或 stm32mp257d-atk-ddr-2GB.dts 中补充 UART/USART 主节点内容, 具体选择依据所购买核心板的 DDR 大小。

示例设备树结构:

stm32mp251.dtsi

```

usart1: serial@40330000 {
    ...
};

...

uart9: serial@402c0000 {
    ...
};

...

lpuart1: serial@46030000 {
    ...
};
    
```

3.3 配置 UART 节点

在配置 UART 设备树节点时,可以参考 USART2 的设备树节点作为示例进行配置。需要特别注意的是,在内核的默认出厂源码中,usart1 和 usart2 被设置为调试串口,分别对应 serial1 和 serial0。为了保证调试功能的稳定性,不建议修改这两个串口的默认配置。对于其他 UART 的配置方式,可以直接参考 USART2 的设备树写法进行修改。

stm32mp257d-atk-ddr-1GB.dts 或 stm32mp257d-atk-ddr-2GB.dts

```
...

aliases {
    ...
    serial0 = &usart2; // 默认将 USART2 定义为 serial0, 不建议修改
    ...
};

...

&usart2 {
    pinctrl-names = "default", "idle", "sleep"; // 定义三种引脚复用状态
    pinctrl-0 = <&usart2_pins_a>; // 默认模式下的引脚配置
    pinctrl-1 = <&usart2_idle_pins_a>; // 空闲模式下的引脚配置
    pinctrl-2 = <&usart2_sleep_pins_a>; // 睡眠模式下的引脚配置
    /delete-property/dmas; // 删除 dmas 属性
    /delete-property/dma-names; // 删除 dma-names 属性
    status = "okay"; // 启用 USART2
};

...
```

关键配置解释:

1. serial0

将 USART2 定义为 serial0, 对应设备/dev/ttySTM0。

```
root@ATK-DLMP257:~# ls /dev/ttySTM0
/dev/ttySTM0
root@ATK-DLMP257:~#
```

图 3.3.1 串口对应得设备文件

2. pinctrl-names = "default", "idle", "sleep";

定义三种引脚复用状态: 默认、空闲和睡眠模式。

3. pinctrl-0、pinctrl-1、pinctrl-2

分别对应不同模式下的引脚配置。通过 <&usart2_pins_a>、<&usart2_idle_pins_a> 和 <&usart2_sleep_pins_a> 来引用具体的引脚配置。

4. /delete-property/dmas 和 /delete-property/dma-names



删除 `dmass` 和 `dma-names` 属性, 表示此配置不使用 DMA。通常 DMA 是用来加速数据传输的, 如果不需要, 可以删除这些属性来简化配置。

5. `status = "okay"`

启用 USART2, 表示该外设被激活并且可以使用。

3.4 修改引脚复用配置

在 `stm32mp25-pinctrl-atk-ddr-2GB.dtsi` 或 `stm32mp25-pinctrl-atk-ddr-1GB.dtsi` 文件中添加 UART 引脚的 `pinctrl` 描述, 并将其复用功能设置为 `AFx`。具体的 `AFx` 值需要根据 STM32MP257D 的数据手册或《【正点原子】ATK-CLMP257B 核心板接口数据手册》进行查询, 以确认正确的复用功能编号。

`stm32mp25-pinctrl-atk-ddr-2GB.dtsi` 或 `stm32mp25-pinctrl-atk-ddr-1GB.dtsi`

```
usart2_pins_a: usart2-0 {
    pins1 {
        pinmux = <STM32_PINMUX('A', 4, AF6)>; /* USART2_TX */
        bias-disable;
        drive-push-pull;
        slew-rate = <0>;
    };
    pins2 {
        pinmux = <STM32_PINMUX('A', 8, AF8)>; /* USART2_RX */
        bias-pull-up;
    };
};

usart2_idle_pins_a: usart2-idle-0 {
    pins1 {
        pinmux = <STM32_PINMUX('A', 4, ANALOG)>; /* USART2_TX */
    };
    pins2 {
        pinmux = <STM32_PINMUX('A', 8, AF8)>; /* USART2_RX */
        bias-pull-up;
    };
};

usart2_sleep_pins_a: usart2-sleep-0 {
    pins {
        pinmux = <STM32_PINMUX('A', 4, ANALOG)>; /* USART2_TX */
        <STM32_PINMUX('A', 8, ANALOG)>; /* USART2_RX */
    };
};
```

如果我们需要将其他引脚复用为 UART, 只需参考上述 USART2 的写法进行修改。除 `AFx` 功能编号需要根据具体引脚的定义进行调整外, `ANALOG` 模式的编号可按示例保持不变。。

3.5 UART 功能调试

核心板启动后，进入文件系统的 `/dev` 目录，可以看到不同串口对应的设备文件，分别为 `ttySTM0`、`ttySTM1` 和 `ttySTM2`。这些设备文件与串口的序号对应关系已在前文说明，大家可以直接通过这些设备文件使用串口功能，并通过 `minicom` 等工具进行调试。

```
root@ATK-DLMP257:~# ls /dev/ttySTM*  
/dev/ttySTM0 /dev/ttySTM1 /dev/ttySTM2  
root@ATK-DLMP257:~# _
```

图 3.5.1 串口调试设备文件

第四章 9 路 UART 复用案例

STM32MP257 最多支持 10 路 UART 接口。其中，需固定分配 1 路 UART 作为 Cortex-A35 核的系统消息输出接口，另可选用 1 路 UART 作为 Cortex-M33 核的系统消息输出接口，以及 1 路 LPUART 接口作为 Cortex-M0+核的系统消息输出接口。因此，实际可供用于数据通信或采集的 UART 接口最多为 9 路。

由于引脚资源存在复用限制，若需同时启用 9 路 UART，则需对部分板载外设功能进行裁剪。为便于用户快速部署，我们提供了一套“最少外设裁剪”的 9 路 UART 参考配置方案。用户可在此基础上，根据具体应用需求，灵活调整保留的外设功能，从而进一步优化 UART 资源的使用策略。

由于 STM32MP257 的 1GB DDR 版本与 2GB DDR 版本在内核设备树中的配置差异较小，接下来的 UART 复用相关案例将以 2GB DDR 版本为基础进行说明和修改示例。如需在 1GB DDR 版本的 STM32MP257 核心板上进行相同配置，可参考相应操作步骤进行调整，修改方法基本一致。

以下是系统通信文件中的 UART 节点与实际串口资源之间的对应关系：

系统通信文件	实际串口资源	串口发送引脚	串口接收引脚
/dev/ttySTM0	USART2	PA4	PA8
/dev/ttySTM1	USART1	PG14	PG15
/dev/ttySTM2	USART3	PI6	PI7
/dev/ttySTM3	UART4	PB7	PB6
/dev/ttySTM4	UART5	PG9	PG10
/dev/ttySTM5	USART6	PF13	PF14
/dev/ttySTM6	UART7	PD3	PD0
/dev/ttySTM7	UART8	PG3	PB13
/dev/ttySTM8	UART9	PD13	PB14
/dev/ttySTM9	LPUART1	PZ9	PZ4

表格 1 UART 节点与实际串口的对应关系

```

root@ATK-DLMP257:~# ls /dev/ttySTM*
/dev/ttySTM0 /dev/ttySTM2 /dev/ttySTM4 /dev/ttySTM6 /dev/ttySTM8
/dev/ttySTM1 /dev/ttySTM3 /dev/ttySTM5 /dev/ttySTM7 /dev/ttySTM9
root@ATK-DLMP257:~# dmesg | grep ttySTM*
[ 0.000000] Kernel command line: root=PARTUUID=491f6117-415d-4f53-80c9-6e0de54deac6 rootwait rw earlyprintk earlycon console=ttySTM0,115200
[ 2.609814] 400f0000.serial: ttySTM2 at MMIO 0x400f0000 (irq = 62, base_baud = 4000000) is a stm32-usart
[ 2.627119] 40100000.serial: ttySTM3 at MMIO 0x40100000 (irq = 63, base_baud = 4000000) is a stm32-usart
[ 2.645018] 40110000.serial: ttySTM4 at MMIO 0x40110000 (irq = 64, base_baud = 4000000) is a stm32-usart
[ 2.662960] 40220000.serial: ttySTM5 at MMIO 0x40220000 (irq = 65, base_baud = 4000000) is a stm32-usart
[ 2.688895] 402c0000.serial: ttySTM8 at MMIO 0x402c0000 (irq = 66, base_baud = 4000000) is a stm32-usart
[ 2.698857] 40330000.serial: ttySTM1 at MMIO 0x40330000 (irq = 67, base_baud = 1000000) is a stm32-usart
[ 2.716707] 40370000.serial: ttySTM6 at MMIO 0x40370000 (irq = 68, base_baud = 4000000) is a stm32-usart
[ 2.734762] 40380000.serial: ttySTM7 at MMIO 0x40380000 (irq = 69, base_baud = 4000000) is a stm32-usart
[ 2.757527] 46030000.serial: ttySTM9 at MMIO 0x46030000 (irq = 70, base_baud = 1000000) is a stm32-usart
[ 3.539653] 400e0000.serial: ttySTM0 at MMIO 0x400e0000 (irq = 88, base_baud = 4000000) is a stm32-usart
[ 3.554706] printk: console [ttySTM0] enabled
root@ATK-DLMP257:~#

```

图 3.5.1 UART 节点系统消息

4.1 屏蔽串口引脚资源冲突的设备树

在实现 9 路 UART 复用配置之前，需首先注释或禁用内核设备树中部分与 UART 引脚资源存在复用冲突的设备节点。否则，UART 驱动在初始化过程中将因资源冲突而无法正常工作。

以下列出了需屏蔽的关键设备树节点内容，请根据实际设备树文件逐项进行修改，以确保 UART 资源能够正确分配与初始化。



1、屏蔽 arch/arm64/boot/dts/st/stm32mp257d-atk-ddr-2GB-rgb.dts 设备树下的 panel_rgb 设备树节点。

stm32mp257d-atk-ddr-2GB-rgb.dts

```
/ {
    .....
    panel_rgb: panel-rgb {
        compatible = "panel-dpi";
        pinctrl-names = "default", "sleep";
        pinctrl-0 = <&ltdc_pins_a>;
        pinctrl-1 = <&ltdc_sleep_pins_a>;
        backlight = <&panel_rgb_backlight>;
        power-supply = <&vdd_5v>;
        status = "disabled"; // add it. - alientek
        .....
    };
};
```

2、屏蔽 arch/arm64/boot/dts/st/stm32mp257d-atk-ddr-2GB.dts 设备树下与 9 路 UART 复用引脚资源冲突的设备树节点。

stm32mp257d-atk-ddr-2GB.dts

```
/ {
    .....
    sound {
        compatible = "audio-graph-card";
        label = "es8388-audio";
        dais = <&sai1a_port &sai1b_port>;
        widgets =
            "Headphone", "Headphone Jack",
            "Speaker", "Ext Spk",
            "Microphone", "Mic Jack";
        routing =
            "Playback", "MCLK",
            "Capture", "MCLK",
            "Headphone Jack", "LOUT1",
            "Headphone Jack", "ROUT1",
            "Ext Spk", "LOUT2",
            "Ext Spk", "ROUT2",
            "LINPUT1", "Mic Jack",
            "LINPUT2", "Mic Jack",
            "RINPUT2", "Mic Jack",
            "Mic Jack", "Mic Bias";
        status = "disabled"; // add it. - alientek
    };
};
```



```

.....
};
.....
&csi {
    vdd-supply = <&scmi_vddcore>;
    vdda18-supply = <&scmi_v1v8>;
    status = "disabled"; // add it. - alientek
    ports {
        #address-cells = <1>;
        #size-cells = <0>;
        port@0 {
            reg = <0>;
            csi_sink: endpoint {
                remote-endpoint = <&imx335_ep>;
                data-lanes = <0 1>;
                bus-type = <4>;
            };
        };

        port@1 {
            reg = <1>;
            csi_source: endpoint {
                remote-endpoint = <&dcmiip_0>;
            };
        };
    };
};
.....
&i2c4 {
.....
    es8388: es8388@11 {
        compatible = "alientek,es8388";
        reg = <0x11>;
        #sound-dai-cells = <0>;
        spk-con-gpios = <&gpiod 12 GPIO_ACTIVE_LOW>;
        hp-det-gpios = <&gpiod 13 GPIO_ACTIVE_LOW>;
        clocks = <&sai1b>;
        clock-names = "MCLK";
        DVDD-supply = <&vdd_3v3>;
        PVDD-supply = <&vdd_3v3>;
        AVDD-supply = <&vdd_3v3>;
        HPVDD-supply = <&vdd_3v3>;
        status = "disabled"; // add it. - alientek
    };
};
.....

```

```

.....
}
.....
};
.....
&i2c8 {
    pinctrl-names = "default", "sleep";
    pinctrl-0 = <&i2c8_pins_a>;
    pinctrl-1 = <&i2c8_sleep_pins_a>;
    status = "disabled"; // add it. - alientek
    /delete-property/dmas;
    /delete-property/dma-names;
};
.....
&m0_rproc {
    mbox-names = <&ipcc2 0>, <&ipcc2 1>, <&ipcc2 2>;
    mbox-names = "rx", "tx", "shutdown";
    memory-region = <&cm0_cube_fw>, <&cm0_cube_data>;
    clocks = <&rcc CK_CPU3>,
            <&rcc CK_CPU3_AM>,
            <&rcc CK_LPUART1_C3>,
            <&rcc CK_KER_LPUART1>,
            <&rcc CK_LPUART1_AM>,
            <&scmi_clk CK_SCMI_IPCC2>,
            <&scmi_clk CK_SCMI_IPCC2_AM>;
    status = "disabled"; // add it. - alientek
};
.....

```

4.2 修改系统支持串口实例数量

在 STM32MP257 的出厂内核源码中, UART 驱动默认最多仅支持初始化 9 个实例对象, 对应设备节点为 /dev/ttySTM0 至 /dev/ttySTM8。然而, 在当前应用场景中, 系统需同时启用 10 路 UART 接口, 因此需对 UART 驱动进行相应修改, 以支持更多的串口实例注册和初始化操作, 从而满足实际使用需求。

在内核源码中, 定位至文件路径 `drivers/tty/serial/stm32-usart.h`, 可找到宏定义 `STM32_MAX_PORTS`。为支持 10 路 UART 接口, 只需将该宏所关联的数值修改为 10, 即可扩展驱动对串口实例的支持范围:

```
#define STM32_MAX_PORTS 10
```

完成修改后, 重新编译内核, 即可实现最多初始化 10 个 UART 实例的能力。

```

213 #define STM32_SERIAL_NAME "ttySTM"
214 // #define STM32_MAX_PORTS 9
215 #define STM32_MAX_PORTS 10 /* alientek */

```

图 4.2.1 修改 UART 驱动初始化 UART 实例数量

4.3 USART1 (/dev/ttySTM1)

在系统默认配置中, USART1 被预设为 Cortex-M33 核的系统消息输出专用接口。如需将该串口作为普通 UART 接口使用, 需对设备树进行相应调整。

请在以下设备树文件中:

arch/arm64/boot/dts/st/stm32mp257d-atk-ddr-2GB.dts

找到并禁用 m33_rproc 相关设备节点。禁用该节点后, 系统将不再保留该串口用于 M33 核输出, 从而可将其配置为普通串口用于数据通信或其他功能。

stm32mp257d-atk-ddr-2GB.dts

```
&m33_rproc {
    mboxs = <&ipcc1 0x100>, <&ipcc1 0x101>, <&ipcc1 2>;
    mbox-names = "vq0", "vq1", "shutdown";
    memory-region = <&cm33_cube_fw>, <&cm33_cube_data>,
        <&ipc_shmem_1>, <&vdev0vring0>,
        <&vdev0vring1>, <&vdev0buffer>,
        <&cm33_sram2>;
    st,syscfg-nsvtor = <&a35ss_syscfg 0xa8 0xffffffff80>;
    status = "disabled";
};
```

完成设备树文件修改后, 保存并重新编译内核源码以生成更新后的设备树二进制文件(.dtb)。将该设备树文件用于开发板启动后, 可通过以下命令验证串口节点是否成功创建:

```
ls /dev/ttySTM1
```

若节点存在, 表示串口已正确初始化。此时, 可进行串口功能测试, 确认其收发数据是否正常。

4.4 USART2 (/dev/ttySTM0)

系统默认配置, USART2 作为 Cortex-A35 核系统消息输出串口使用, 不需要修改。

4.5 USART3 (/dev/ttySTM2)

1、在 arch/arm64/boot/dts/st/stm32mp257d-atk-ddr-2GB.dts 设备树文件中, 补充与 USART3 相关的设备树节点。

stm32mp257d-atk-ddr-2GB.dts

```
.....
/ {
    aliases {
        .....
        serial2 = &usart3;
        .....
    };
};
.....
&usart3 {
```

```
pinctrl-names = "default", "idle", "sleep";
pinctrl-0 = <&usart3_pins_a>;
pinctrl-1 = <&usart3_idle_pins_a>;
pinctrl-2 = <&usart3_sleep_pins_a>;
status = "okay";

};

.....
```

2、在 arch/arm64/boot/dts/st/stm32mp25-pinctrl-atk-ddr-2GB.dtsi 设备树下补充 USART3 相关的引脚定义。

stm32mp25-pinctrl-atk-ddr-2GB.dtsi

```
.....

usart3_pins_a: usart3-0 {
    pins1 {
        pinmux = <STM32_PINMUX('T', 6, AF6)>; /* USART3_TX */
        bias-disable;
        drive-push-pull;
        slew-rate = <0>;
    };
    pins2 {
        pinmux = <STM32_PINMUX('T', 7, AF6)>; /* USART3_RX */
        bias-pull-up;
    };
};

usart3_idle_pins_a: usart3-idle-0 {
    pins1 {
        pinmux = <STM32_PINMUX('T', 6, ANALOG)>; /* USART3_TX */
    };
    pins2 {
        pinmux = <STM32_PINMUX('T', 7, AF6)>; /* USART3_RX */
        bias-pull-up;
    };
};

usart3_sleep_pins_a: usart3-sleep-0 {
    pins {
        pinmux = <STM32_PINMUX('T', 6, ANALOG)>, /* USART3_TX */
        <STM32_PINMUX('T', 7, ANALOG)>; /* USART3_RX */
    };
};

.....
```

3、完成设备树文件修改后，保存并重新编译内核源码以生成更新后的设备树二进制文件（.dtb）。将该设备树文件用于开发板启动后，可通过以下命令验证串口节点是否成功创建：



```
ls /dev/ttySTM2
```

若节点存在，表示串口已正确初始化。此时，可进行串口功能测试，确认其收发数据是否正常。

4.6 UART4 (/dev/ttySTM3)

1、在 arch/arm64/boot/dts/st/stm32mp257d-atk-ddr-2GB.dts 设备树文件中，补充与 UART4 相关的设备树节点。

stm32mp257d-atk-ddr-2GB.dts

```
.....
/ {
    aliases {
        .....
        serial3 = &uart4;
        .....
    };
};
.....
&uart4 {
    pinctrl-names = "default", "idle", "sleep";
    pinctrl-0 = <&uart4_pins_a>;
    pinctrl-1 = <&uart4_idle_pins_a>;
    pinctrl-2 = <&uart4_sleep_pins_a>;
    status = "okay";
};
.....
```

2、在 arch/arm64/boot/dts/st/stm32mp25-pinctrl-atk-ddr-2GB.dtsi 设备树下补充 UART4 相关的引脚定义。

stm32mp25-pinctrl-atk-ddr-2GB.dtsi

```
.....
uart4_pins_a: uart4-0 {
    pins1 {
        pinmux = <STM32_PINMUX('B', 7, AF3)>; /* UART4_TX */
        bias-disable;
        drive-push-pull;
        slew-rate = <0>;
    };
    pins2 {
        pinmux = <STM32_PINMUX('B', 6, AF3)>; /* UART4_RX */
        bias-pull-up;
    };
};
```

```
uart4_idle_pins_a: uart4-idle-0 {
    pins1 {
        pinmux = <STM32_PINMUX('B', 7, ANALOG)>; /* UART4_TX */
    };
    pins2 {
        pinmux = <STM32_PINMUX('B', 6, AF3)>; /* UART4_RX */
        bias-pull-up;
    };
};

uart4_sleep_pins_a: uart4-sleep-0 {
    pins {
        pinmux = <STM32_PINMUX('B', 7, ANALOG)>; /* UART4_TX */
        <STM32_PINMUX('B', 6, ANALOG)>; /* UART4_RX */
    };
};

.....
```

3、完成设备树文件修改后，保存并重新编译内核源码以生成更新后的设备树二进制文件（.dtb）。将该设备树文件用于开发板启动后，可通过以下命令验证串口节点是否成功创建：

```
ls /dev/ttySTM3
```

若节点存在，表示串口已正确初始化。此时，可进行串口功能测试，确认其收发数据是否正常。

4.1 UART5 (/dev/ttySTM4)

1、在 arch/arm64/boot/dts/st/stm32mp257d-atk-ddr-2GB.dts 设备树文件中，补充与 UART5 相关的设备树节点。

stm32mp257d-atk-ddr-2GB.dts

```
.....
/ {
    aliases {
        .....
        serial4 = &uart5;
        .....
    };
};

.....
&uart5 {
    pinctrl-names = "default", "idle", "sleep";
    pinctrl-0 = <&uart5_pins_a>;
    pinctrl-1 = <&uart5_idle_pins_a>;
    pinctrl-2 = <&uart5_sleep_pins_a>;
    status = "okay";
};
```



.....

2、在 arch/arm64/boot/dts/st/stm32mp25-pinctrl-atk-ddr-2GB.dtsi 设备树下补充 UART5 相关的引脚定义。

stm32mp25-pinctrl-atk-ddr-2GB.dtsi

.....

```
uart5_pins_a: uart5-0 {
    pins1 {
        pinmux = <STM32_PINMUX('G', 9, AF5)>; /* UART5_TX */
        bias-disable;
        drive-push-pull;
        slew-rate = <0>;
    };
    pins2 {
        pinmux = <STM32_PINMUX('G', 10, AF5)>; /* UART5_RX */
        bias-pull-up;
    };
};

uart5_idle_pins_a: uart5-idle-0 {
    pins1 {
        pinmux = <STM32_PINMUX('G', 9, ANALOG)>; /* UART5_TX */
    };
    pins2 {
        pinmux = <STM32_PINMUX('G', 10, AF5)>; /* UART5_RX */
        bias-pull-up;
    };
};

uart5_sleep_pins_a: uart5-sleep-0 {
    pins {
        pinmux = <STM32_PINMUX('G', 9, ANALOG)>, /* UART5_TX */
        <STM32_PINMUX('G', 10, ANALOG)>; /* UART5_RX */
    };
};
```

.....

3、完成设备树文件修改后，保存并重新编译内核源码以生成更新后的设备树二进制文件（.dtb）。将该设备树文件用于开发板启动后，可通过以下命令验证串口节点是否成功创建：

```
ls /dev/ttySTM4
```

若节点存在，表示串口已正确初始化。此时，可进行串口功能测试，确认其收发数据是否正常。



4.1 USART6 (/dev/ttySTM5)

1、在 arch/arm64/boot/dts/st/stm32mp257d-atk-ddr-2GB.dts 设备树文件中，补充与 USART6 相关的设备树节点。

stm32mp257d-atk-ddr-2GB.dts

```

.....
/ {
    aliases {
        .....
        serial5 = &usart6;
        .....
    };
};
.....
&usart6 {
    pinctrl-names = "default", "idle", "sleep";
    pinctrl-0 = <&usart6_pins_a>;
    pinctrl-1 = <&usart6_idle_pins_a>;
    pinctrl-2 = <&usart6_sleep_pins_a>;
    status = "okay";
};
.....

```

2、在 arch/arm64/boot/dts/st/stm32mp25-pinctrl-atk-ddr-2GB.dtsi 设备树下补充 USART6 相关的引脚定义。

stm32mp25-pinctrl-atk-ddr-2GB.dtsi

```

.....
usart6_pins_a: usart6-0 {
    pins1 {
        pinmux = <STM32_PINMUX('F', 13, AF3)>; /* USART6_TX */
        bias-disable;
        drive-push-pull;
        slew-rate = <0>;
    };
    pins2 {
        pinmux = <STM32_PINMUX('F', 14, AF3)>; /* USART6_RX */
        bias-pull-up;
    };
};

usart6_idle_pins_a: usart6-idle-0 {
    pins1 {
        pinmux = <STM32_PINMUX('F', 13, ANALOG)>; /* USART6_TX */
    };
};

```

```

pins2 {
    pinmux = <STM32_PINMUX('F', 14, AF3)>; /* USART6_RX */
    bias-pull-up;
};

};

uart6_sleep_pins_a: uart6-sleep-0 {
    pins {
        pinmux = <STM32_PINMUX('F', 13, ANALOG)>, /* USART6_TX */
        <STM32_PINMUX('F', 14, ANALOG)>; /* USART6_RX */
    };
};

.....

```

3、完成设备树文件修改后，保存并重新编译内核源码以生成更新后的设备树二进制文件（.dtb）。将该设备树文件用于开发板启动后，可通过以下命令验证串口节点是否成功创建：

```
ls /dev/ttySTM5
```

若节点存在，表示串口已正确初始化。此时，可进行串口功能测试，确认其收发数据是否正常。

4.1 UART7 (/dev/ttySTM6)

1、在 arch/arm64/boot/dts/st/stm32mp257d-atk-ddr-2GB.dts 设备树文件中，补充与 UART7 相关的设备树节点。

stm32mp257d-atk-ddr-2GB.dts

```

.....
/ {
    aliases {
        .....
        serial6 = &uart7;
        .....
    };
};

.....
&uart7 {
    pinctrl-names = "default", "idle", "sleep";
    pinctrl-0 = <&uart7_pins_a>;
    pinctrl-1 = <&uart7_idle_pins_a>;
    pinctrl-2 = <&uart7_sleep_pins_a>;
    status = "okay";
};

.....

```

2、在 arch/arm64/boot/dts/st/stm32mp25-pinctrl-atk-ddr-2GB.dtsi 设备树下补充 UART7 相关的引脚定义。



stm32mp25-pinctrl-atk-ddr-2GB.dtsi

```

.....
uart7_pins_a: uart7-0 {
    pins1 {
        pinmux = <STM32_PINMUX('D', 3, AF6)>; /* UART7_TX */
        bias-disable;
        drive-push-pull;
        slew-rate = <0>;
    };
    pins2 {
        pinmux = <STM32_PINMUX('D', 0, AF6)>; /* UART7_RX */
        bias-pull-up;
    };
};

uart7_idle_pins_a: uart7-idle-0 {
    pins1 {
        pinmux = <STM32_PINMUX('D', 3, ANALOG)>; /* UART7_TX */
    };
    pins2 {
        pinmux = <STM32_PINMUX('D', 0, AF6)>; /* UART7_RX */
        bias-pull-up;
    };
};

uart7_sleep_pins_a: uart7-sleep-0 {
    pins {
        pinmux = <STM32_PINMUX('D', 3, ANALOG)>, /* UART7_TX */
        <STM32_PINMUX('D', 0, ANALOG)>; /* UART7_RX */
    };
};
.....

```

3、完成设备树文件修改后，保存并重新编译内核源码以生成更新后的设备树二进制文件（.dtb）。将该设备树文件用于开发板启动后，可通过以下命令验证串口节点是否成功创建：

```
ls /dev/ttySTM6
```

若节点存在，表示串口已正确初始化。此时，可进行串口功能测试，确认其收发数据是否正常。

4.1 UART8 (/dev/ttySTM7)

1、在 arch/arm64/boot/dts/st/stm32mp257d-atk-ddr-2GB.dts 设备树文件中，补充与 UART8 相关的设备树节点。

stm32mp257d-atk-ddr-2GB.dts



```

.....
/ {
    aliases {
        .....
        serial7 = &uart8;
        .....
    };
};
.....
&uart8 {
    pinctrl-names = "default", "idle", "sleep";
    pinctrl-0 = <&uart8_pins_a>;
    pinctrl-1 = <&uart8_idle_pins_a>;
    pinctrl-2 = <&uart8_sleep_pins_a>;
    status = "okay";
};
.....

```

2、在 arch/arm64/boot/dts/st/stm32mp25-pinctrl-atk-ddr-2GB.dtsi 设备树下补充 UART8 相关的引脚定义。

stm32mp25-pinctrl-atk-ddr-2GB.dtsi

```

.....
uart8_pins_a: uart8-0 {
    pins1 {
        pinmux = <STM32_PINMUX('G', 3, AF3)>; /* UART8_TX */
        bias-disable;
        drive-push-pull;
        slew-rate = <0>;
    };
    pins2 {
        pinmux = <STM32_PINMUX('B', 13, AF5)>; /* UART8_RX */
        bias-pull-up;
    };
};

uart8_idle_pins_a: uart8-idle-0 {
    pins1 {
        pinmux = <STM32_PINMUX('G', 3, ANALOG)>; /* UART8_TX */
    };
    pins2 {
        pinmux = <STM32_PINMUX('B', 13, AF5)>; /* UART8_RX */
        bias-pull-up;
    };
};

```



```

uart8_sleep_pins_a: uart8-sleep-0 {
    pins {
        pinmux = <STM32_PINMUX('G', 3, ANALOG)>, /* UART8_TX */
        <STM32_PINMUX('B', 13, ANALOG)>; /* UART8_RX */
    };
};
.....

```

3、完成设备树文件修改后，保存并重新编译内核源码以生成更新后的设备树二进制文件（.dtb）。将该设备树文件用于开发板启动后，可通过以下命令验证串口节点是否成功创建：

```
ls /dev/ttySTM8
```

若节点存在，表示串口已正确初始化。此时，可进行串口功能测试，确认其收发数据是否正常。

4.1 UART9 (/dev/ttySTM8)

1、在 arch/arm64/boot/dts/st/stm32mp257d-atk-ddr-2GB.dts 设备树文件中，补充与 UART9 相关的设备树节点。

stm32mp257d-atk-ddr-2GB.dts

```

.....
/ {
    aliases {
        .....
        serial8 = &uart9;
        .....
    };
};
.....
&uart9 {
    pinctrl-names = "default", "idle", "sleep";
    pinctrl-0 = <&uart9_pins_a>;
    pinctrl-1 = <&uart9_idle_pins_a>;
    pinctrl-2 = <&uart9_sleep_pins_a>;
    status = "okay";
};
.....

```

2、在 arch/arm64/boot/dts/st/stm32mp25-pinctrl-atk-ddr-2GB.dtsi 设备树下补充 UART9 相关的引脚定义。

stm32mp25-pinctrl-atk-ddr-2GB.dtsi

```

.....
uart9_pins_a: uart9-0 {
    pins1 {
        pinmux = <STM32_PINMUX('D', 13, AF6)>; /* UART9_TX */
    };
};
.....

```

```

        bias-disable;
        drive-push-pull;
        slew-rate = <0>;
    };
    pins2 {
        pinmux = <STM32_PINMUX('B', 14, AF6)>; /* UART9_RX */
        bias-pull-up;
    };
};

uart9_idle_pins_a: uart9-idle-0 {
    pins1 {
        pinmux = <STM32_PINMUX('D', 13, ANALOG)>; /* UART9_TX */
    };
    pins2 {
        pinmux = <STM32_PINMUX('B', 14, AF6)>; /* UART9_RX */
        bias-pull-up;
    };
};

uart9_sleep_pins_a: uart9-sleep-0 {
    pins {
        pinmux = <STM32_PINMUX('D', 13, ANALOG)>; /* UART9_TX */
        <STM32_PINMUX('B', 14, ANALOG)>; /* UART9_RX */
    };
};

.....

```

3、完成设备树文件修改后，保存并重新编译内核源码以生成更新后的设备树二进制文件（.dtb）。将该设备树文件用于开发板启动后，可通过以下命令验证串口节点是否成功创建：

```
ls /dev/ttySTM8
```

若节点存在，表示串口已正确初始化。此时，可进行串口功能测试，确认其收发数据是否正常。

4.1 LPUART1 (/dev/ttySTM9)

1、在 arch/arm64/boot/dts/st/stm32mp257d-atk-ddr-2GB.dts 设备树文件中，补充与 LPUART 1 相关的设备树节点。

stm32mp257d-atk-ddr-2GB.dts

```

.....
/ {
    aliases {
        .....
        serial9= &lpuart1;
        .....
    }
}

```



```

    };
};
.....
&lpuart1 {
    pinctrl-names = "default";
    pinctrl-0 = <&lpuart1_pins_a>;
    /delete-property/dmas;
    /delete-property/dma-names;
    uart-has-rtsscts;
    status = "okay";
};
.....

```

2、在 arch/arm64/boot/dts/st/stm32mp25-pinctrl-atk-ddr-2GB.dtsi 设备树下补充 LPUART1 相关的引脚定义。

stm32mp25-pinctrl-atk-ddr-2GB.dtsi

```

.....
&pinctrl_z {
    .....
    lpuart1_pins_a: lpuart1-0 {
        pins1 {
            pinmux = <STM32_PINMUX('Z', 9, AF6)>; /* LPUART1_TX */
            bias-disable;
            drive-push-pull;
            slew-rate = <0>;
        };
        pins2 {
            pinmux = <STM32_PINMUX('Z', 4, AF6)>; /* LPUART1_RX */
            bias-pull-up;
        };
    };
};
.....
};

```

3、完成设备树文件修改后，保存并重新编译内核源码以生成更新后的设备树二进制文件（.dtb）。将该设备树文件用于开发板启动后，可通过以下命令验证串口节点是否成功创建：

```
ls /dev/ttySTM9
```

若节点存在，表示串口已正确初始化。此时，可进行串口功能测试，确认其收发数据是否正常。

第五章 多路 ADC 复用

STM32MP257 支持 ADC1、ADC2、ADC3，总共提供 23 个通道。尽管 STM32MP257 的 ADC 支持通过采集差分信号来提高精度，但为了获得更高的 ADC 数据精度，通常建议选择外部 ADC 芯片。如果需要使用 STM32MP257 的其他 ADC 采集引脚，可以按照以下步骤进行配置。

5.1 确认引脚所属 ADC 控制器

首先，确认所使用的引脚属于 ADC1、ADC2 还是 ADC3 控制器。需要参考 STM32MP257 数据手册，例如：

- PC5 引脚可用作：
 - ADC1_INP10: ADC1 控制器的第 10 正向差分信号输入通道
 - ADC2_INP10: ADC2 控制器的第 10 正向差分信号输入通道
 - ADC3_INP10: ADC3 控制器的第 10 正向差分信号输入通道
- PC3 引脚可用作：
 - ADC1_INP12: ADC1 控制器的第 12 正向差分信号输入通道
 - 其他功能类似

Pin number			Pin name (function after reset)	Pin type	I/O structure	Notes	Alternate functions	Additional functions
VFBGA361	VFBGA424	TFBGA436						
V9	AB14	AA12	PC0	I/O	TT	(1)	LPTIM1_CH1, SPI6_SCK, SAI3_MCLK_B, USART6_TX, DCMI_D0/PSSI_D0/DCMIPP_D0, ETH2_MII_RX_CLK/ETH2_RMII_REF_CLK, ETH1_MII_TX_CLK, ETH1_RGMII_GTX_CLK, LCD_G7, EVENTOUT	-
V10	AG14	V11	PC1	I/O	TT_f	(1)	SPI3_MOSI/I2S3_SDO, USART2_TX, I2C7_SCL, ETH1_MII_TXD1/ETH1_RGMII_TXD1/ETH1_RMII_TXD1, EVENTOUT	-
T9	AE12	Y11	PC2	I/O	TT	(1)	SPI8_MOSI, LPTIM2_IN1, SAI4_MCLK_B, MDF1_SDI3, USART2_RTS/USART2_DE, ETH1_MII_RXD1/ETH1_RGMII_RXD1/ETH1_RMII_RXD1, EVENTOUT	-
U4	AA9	W9	PC3	I/O	TT_a	(5)	LPTIM1_IN2, SPI3_NSS/I2S3_WS, SPI6_RDY, USART6_RTS/USART6_DE, FDCAN2_TX, ETH2_MII_RX_DV/ETH2_RGMII_RX_CTL/ETH2_RMII_CRD_DV, ETH1_MII_RX_ER, LCD_G6, DCMI_D3/PSSI_D3/DCMIPP_D3, EVENTOUT	ADC1_INP12, ADC1_INN10, ADC2_INP12, ADC2_INN10, ADC3_INP12, ADC3_INN10, TAMP_IN3
V3	AC9	V9	PC4	I/O	TT	(5)	SPI6_MISO, SAI3_FS_B, ETH2_MII_TX_EN/ETH2_RGMII_TX_CTL/ETH2_RMII_TX_EN, ETH1_RGMII_CLK125, LCD_R0, EVENTOUT	TAMP_IN1
T7	AD10	U9	PC5	I/O	TT_af	(5)	SPDIFRX1_IN1, MDF1_SDI1, TIM8_CH1N, I2C4_SDA, ETH2_MDIO, ETH1_MII_COL, FMC_A25, ETH1_PPS_OUT, LCD_DE, EVENTOUT	ADC1_INP10, ADC2_INP10, ADC3_INP10, TAMP_IN6

图 5.1.1 STM32MP257 数据手册

5.2 配置设备树中的 ADC 主节点

设备树文件位于 stm32mp251.dtsi，需要在 stm32mp257d-atk-ddr-1GB.dts 或 stm32mp257d-atk-ddr-2GB.dts 中补充 ADC 主节点内容，具体选择依据所购买核心板的 DDR 大小。

示例设备树结构：

```
stm32mp251.dtsi

adc_12: adc@404e0000 {
    ...
    adc1: adc@0 {
```



```

        ...
    };
    adc2: adc@100 {
        ...
    };
};

adc_3: adc@404f0000 {
    ...
    adc3: adc@0 {
        ...
    };
};

```

5.3 配置 ADC 通道

以 ADC1 的第 15 通道（ADC1_INP15）为例，对设备树中的 ADC 节点进行配置：

[stm32mp257d-atk-ddr-1GB.dts](#) 或 [stm32mp257d-atk-ddr-2GB.dts](#)

```

&adc_12 {
    pinctrl-names = "default";
    pinctrl-0 = <&adc1_in15_pins_a>;
    vdda-supply = <&vdda_1v8>;
    vref-supply = <&vddref_1v8>;
    status = "okay";

    adc1: adc@0 {
        #address-cells = <1>;
        #size-cells = <0>;
        status = "okay";

        channel@15 {
            reg = <15>;                                /* 使用 ADC1 的第 15 通道 */
            st,min-sample-time-ns = <10000>;           /* 最小采样时间为 10µs */
        };
    };
};

```

关键配置解释：

- 1. pinctrl-0 = <&adc1_in15_pins_a>;**
引用名为 adc1_in15_pins_a 的引脚配置。
- 2. adc1: adc@0**
定义 ADC1 的主配置节点，@0 表示地址。
- 3. channel@15**
配置 ADC1 的第 15 通道：
 - reg = <15>; 指定使用第 15 通道。



- `st,min-sample-time-ns = <10000>`; 设置最小采样时间为 10 微秒。

5.4 修改引脚复用配置

在 `stm32mp25-pinctrl-atk-ddr-2GB.dtsi` 或 `stm32mp25-pinctrl-atk-ddr-1GB.dtsi` 文件中, 添加 ADC 引脚的 `pinctrl` 描述, 并将其复用功能设置为 “ANALOG”。

`stm32mp25-pinctrl-atk-ddr-2GB.dtsi` 或 `stm32mp25-pinctrl-atk-ddr-1GB.dtsi`

```
adc1_in15_pins_a: adc1-in15 {
    pins {
        pinmux = <STM32_PINMUX('B', 15, ANALOG)>;
    };
};
```

5.5 更新设备树以支持多个 ADC 通道

在配置多个 ADC 通道时, 应更新主设备树文件以包含所有相关引脚的配置。例如, 将 PC3 和 PB15 分别复用为 ADC1_INP12 和 ADC1_INP15, PC6 复用为 ADC2_INP6, PC5 复用为 ADC3_INP10。

步骤如下: 。

1、修改引脚复用配置

- 检查是否有其他设备树节点使用了相同的引脚。
- 如果发现冲突, 必须注释或禁用相关的设备树节点, 以避免引脚复用冲突。

`stm32mp25-pinctrl-atk-ddr-2GB.dtsi` 或 `stm32mp25-pinctrl-atk-ddr-1GB.dtsi`

```
adc1_in15_pins_a: adc1-in15 {
    pins {
        pinmux = <STM32_PINMUX('B', 15, ANALOG)>;
    };
};

adc1_in12_pins_a: adc1-in12 {
    pins {
        pinmux = <STM32_PINMUX('C', 3, ANALOG)>;
    };
};

adc2_in6_pins_a: adc2-in6 {
    pins {
        pinmux = <STM32_PINMUX('C', 6, ANALOG)>;
    };
};

adc3_in10_pins_a: adc3-in10 {
    pins {
        pinmux = <STM32_PINMUX('C', 5, ANALOG)>;
    };
};
```



};

};

2、更新内核设备树

- 配置 ADC 主节点下的通道设置，确保所有需要的 ADC 通道均已正确配置。
- 确认每个 ADC 通道对应的引脚和寄存器地址设置正确无误。

stm32mp257d-atk-ddr-1GB.dts 或 stm32mp257d-atk-ddr-2GB.dts

```

&adc_12 {
    pinctrl-names = "default";
    pinctrl-0 = <&adc1_in12_pins_a>, <&adc1_in15_pins_a>, <&adc2_in6_pins_a>;
    vdda-supply = <&vdda_1v8>;
    vref-supply = <&vddref_1v8>;
    status = "okay";

    adc1: adc@0 {
        #address-cells = <1>;
        #size-cells = <0>;
        status = "okay";

        channel@12 {
            reg = <12>;
            st,min-sample-time-ns = <10000>;
        };

        channel@15 {
            reg = <15>;
            st,min-sample-time-ns = <10000>;
        };
    };

    adc2: adc@100 {
        #address-cells = <1>;
        #size-cells = <0>;
        status = "okay";

        channel@6 {
            reg = <6>;
            st,min-sample-time-ns = <10000>;
        };
    };
};

&adc_3 {
    pinctrl-names = "default";

```

```
pinctrl-0 = <&adc3_in10_pins_a>;
vdda-supply = <&vdda_1v8>;
vref-supply = <&vddref_1v8>;
status = "okay";

adc3: adc@0 {
    #address-cells = <1>;
    #size-cells = <0>;
    status = "okay";

    channel@10 {
        reg = <10>;
        st,min-sample-time-ns = <10000>;
    };
};
```

5.6 ADC 功能测试

核心板启动后，进入文件系统的/sys/bus/iio/devices 目录，可以看到不同 ADC 主控制器下挂载的设备，分别为 iio:device0、iio:device1 和 iio:device2。需要注意的是，ATK-DLMP257B 底板仅支持板载 ADC 输入的测试（PB15 引脚），若需测试其他 ADC 引脚，需自行设计相应的硬件测试方案。

```
cd /sys/bus/iio/devices
ls -l
```

```
root@ATK-DLMP257:/sys/bus/iio/devices# ls -l
total 0
lrwxrwxrwx 1 root root 0 Jan  1 2000 iio:device0 -> ../../../../devices/platform/soc@42080000.rifsc/404e0000.adc/404e0000.adc:adc@0/iio:device0
lrwxrwxrwx 1 root root 0 Jan  1 2000 iio:device1 -> ../../../../devices/platform/soc@42080000.rifsc/404e0000.adc/404e0000.adc:adc@100/iio:device1
lrwxrwxrwx 1 root root 0 Jan  1 2000 iio:device2 -> ../../../../devices/platform/soc@42080000.rifsc/404f0000.adc/404f0000.adc:adc@0/iio:device2
root@ATK-DLMP257:/sys/bus/iio/devices#
```

图 5.6.1 ADC 控制器设备列表

从图 4.6.1 可以看出，各设备对应的 ADC 控制器寄存器地址。在 stm32mp251.dtsi 设备树文件中，ADC1 至 ADC3 主控制器的寄存器地址均已标明，便于快速定位并测量不同 ADC 主控制器通道下的设备文件。

ADC 控制器	控制器寄存器地址
ADC12	adc@404e0000
ADC3	adc@404f0000

配置完成并重启后，可以通过以下命令读取 ADC 数据：。

```
cat /sys/bus/iio/devices/iio:device0/in_voltage12_raw
cat /sys/bus/iio/devices/iio:device0/in_voltage15_raw
cat /sys/bus/iio/devices/iio:device1/in_voltage6_raw
cat /sys/bus/iio/devices/iio:device2/in_voltage10_raw
```

```
root@ATK-DLMP257:/sys/bus/iio/devices# ls
iio:device0 iio:device1 iio:device2
root@ATK-DLMP257:/sys/bus/iio/devices# cat /sys/bus/iio/devices/iio:device0/in_voltage12_raw
1361
root@ATK-DLMP257:/sys/bus/iio/devices# cat /sys/bus/iio/devices/iio:device0/in_voltage15_raw
4071
root@ATK-DLMP257:/sys/bus/iio/devices# cat /sys/bus/iio/devices/iio:device1/in_voltage6_raw
819
root@ATK-DLMP257:/sys/bus/iio/devices# cat /sys/bus/iio/devices/iio:device2/in_voltage10_raw
4095
root@ATK-DLMP257:/sys/bus/iio/devices#
```

图 5.6.2 读取电压采集模拟量

或者使用 watch 命令实时监控所有 ADC 通道：

```
watch -n 1 "echo ADC1_INP12: \$(cat /sys/bus/iio/devices/iio:device0/in_voltage12_raw) &
& echo ADC1_INP15: \$(cat /sys/bus/iio/devices/iio:device0/in_voltage15_raw) && echo ADC2
_INP6: \$(cat /sys/bus/iio/devices/iio:device1/in_voltage6_raw) && echo ADC3_INP10: \$(cat /s
ys/bus/iio/devices/iio:device2/in_voltage10_raw)"
```

```
root@ATK-DLMP257:/sys/bus/iio/devices# watch -n 1 "echo ADC1_INP12: \$(cat /sys/bus/iio/devices/iio:device0/in_voltage12_raw) && echo
ADC1_INP15: \$(cat /sys/bus/iio/devices/iio:device0/in_voltage15_raw) && echo ADC2_INP6: \$(cat /sys/bus/iio/devices/iio:device1/in_
voltage6_raw) && echo ADC3_INP10: \$(cat /sys/bus/iio/devices/iio:device2/in_voltage10_raw)"
root@ATK-DLMP257:/sys/bus/iio/devices#
```

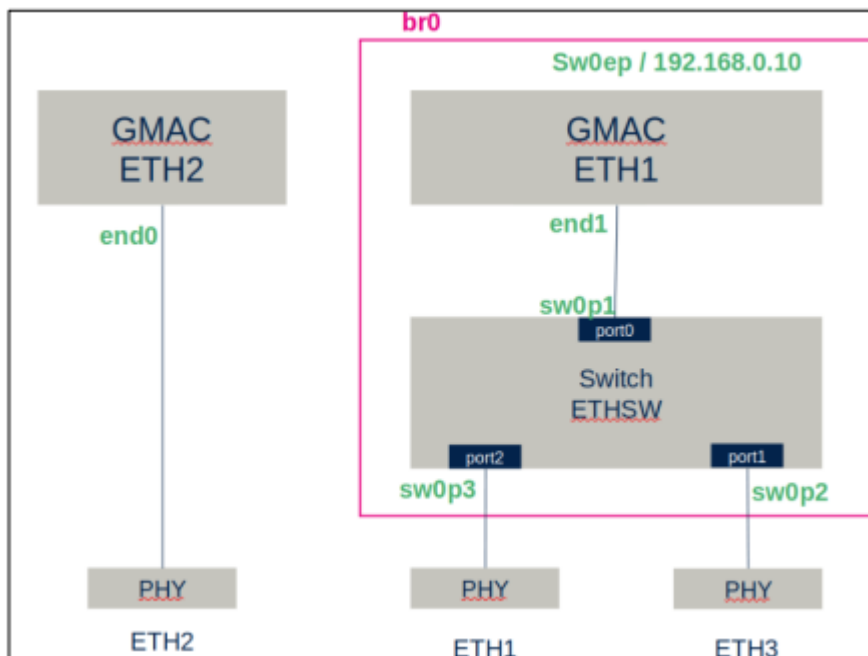
图 5.6.3 watch 指令检测 ADC 通道

```
Every 1.0s: echo ADC1_INP12: \$(cat /sys/bus/iio/devices/iio:device0/in_voltage12_raw) && ec... ATK-DLMP257: Fri Mar 3 16:44:10 2023
ADC1_INP12: 1361
ADC1_INP15: 4072
ADC2_INP6: 818
ADC3_INP10: 4095
```

图 5.6.4 ADC 通道监测结果显示

第六章 ENET 裁剪

由于 STM32MP257 的三个以太网接口 (ETH1、ETH2、ETH3) 本质上是通过两个 GMAC 控制器连接到外部 PHY 芯片实现的, 具体的连接方式可以参考以下提供的网络拓扑图来理解其背后的硬件架构。因此, 本章节将聚焦于如何分别裁剪 ETH1 和 ETH2 的使用。



6.1 单独使用 ETH1 (网口 ENET1)

1、修改 UBoot 设备树

以下内容以修改 STM32MP257 DDR 2GB 核心板版本的设备树文件 arch/arm/dts/stm32mp257d-atk-ddr-2GB.dts 为例进行说明。如果你使用的是 1GB DDR 核心板, 则应对应修改 arch/arm/dts/stm32mp257d-atk-ddr-1GB.dts 文件。

由于两者在本次需要调整的设备树内容上完全一致, 因此此处仅以 2GB 版本为代表进行演示修改, 其他版本可参照相同步骤操作即可。

(1) 修改 eth1 设备树节点内容

arch/arm/dts/stm32mp257d-atk-ddr-2GB.dts

```
&eth1 {
    status = "okay";
    pinctrl-0 = <&eth1_rgmii_pins_a &eth1_mdio_pins_a>;
    pinctrl-1 = <&eth1_rgmii_sleep_pins_a &eth1_mdio_sleep_pins_a>;
    pinctrl-names = "default", "sleep";
    phy-mode = "rgmii-id";
    max-speed = <1000>;
    phy-handle = <&phy1_eth1>;
    st,eth-clk-sel;
    snps,ext-systime;
```



```

mdio1 {
    #address-cells = <1>;
    #size-cells = <0>;
    compatible = "snps,dwmac-mdio";
    phy1_eth1: ethernet-phy@4 {
        compatible = "ethernet-phy-id4f51.e91b";
        reset-gpios = <&gpiof 3 GPIO_ACTIVE_LOW>;
        reset-assert-us = <10000>;
        reset-deassert-us = <80000>;
        reg = <4>;
    };
};
};

```

(2) 屏蔽 eth2 设备树节点

[arch/arm/dts/stm32mp257d-atk-ddr-2GB.dts](#)

```

&eth2 {
    status = "disabled";
    pinctrl-0 = <&eth2_rgmii_pins_a>;
    pinctrl-1 = <&eth2_rgmii_sleep_pins_a>;
    pinctrl-names = "default", "sleep";
    phy-mode = "rgmii-id";
    max-speed = <1000>;
    phy-handle = <&phy1_eth2>;
    st,eth-ptp-from-rcc;

    mdio1 {
        #address-cells = <1>;
        #size-cells = <0>;
        compatible = "snps,dwmac-mdio";
        phy1_eth2: ethernet-phy@1 {
            compatible = "ethernet-phy-id4f51.e91b";
            reset-gpios = <&gpiof 4 GPIO_ACTIVE_LOW>;
            reset-assert-us = <10000>;
            reset-deassert-us = <80000>;
            reg = <1>;
        };
    };
};

```

(3) 屏蔽 switch0 设备节点

[arch/arm/dts/stm32mp257d-atk-ddr-2GB.dts](#)

```

&switch0 {
    status = "disabled";

```

```
pinctrl-0 = <&eth1_rgmii_pins_a>, <&eth3_rgmii_pins_a>;
pinctrl-1 = <&eth1_rgmii_sleep_pins_a>, <&eth3_rgmii_sleep_pins_a>;
pinctrl-names = "default", "sleep";
phy-mode = "rgmii";
st,ethsw-internal-125;

};
```

(4) uboot 终端 ENET1 测试验证

首先将网线插入开发板的 ENET1 网口，启动开发板进入 Uboot 终端，输入以下指令初始化 eth1 端口：

```
setenv ethact eth1@482c0000
setenv ipaddr 192.168.6.120
setenv eth1addr 00:11:22:33:44:66
setenv gatewayip 192.168.6.1
setenv netmask 255.255.255.0
setenv serverip 192.168.6.140
```

这里的 ipaddr、gatewayip、serverip 地址应根据自己实际使用情况来修改。

要测试 ENET1 网口是否工作正常，我们可以直接在 U-Boot 环境中使用 ping 指令进行验证。不过要注意，不能随便 ping 任意 IP 地址！只能 ping 前面通过 setenv serverip 设置好的服务器 IP 地址。

```
STM32MP> ping 192.168.6.140
eth1@482c0000 Waiting for PHY auto negotiation to complete. done
Using eth1@482c0000 device
host 192.168.6.140 is alive
```

图 6.1.1 Uboot 终端测试 ENET1

2、修改 Kernel 设备树

以下内容以修改 STM32MP257 DDR 2GB 核心板版本的设备树文件 arch/arm64/boot/dts/st/stm32mp257d-atk-ddr-2GB.dts 为例进行说明。如果你使用的是 1GB DDR 核心板，则应对应修改 arch/arm64/boot/dts/st/stm32mp257d-atk-ddr-1GB.dts 文件。

由于两者在本次需要调整的设备树内容上完全一致，因此此处仅以 2GB 版本为代表进行演示修改，其他版本可参照相同步骤操作即可。

(1) 修改 eth1 设备树节点内容

arch/arm64/boot/dts/st/stm32mp257d-atk-ddr-2GB.dts

```
&eth1 {
    status = "okay";
    pinctrl-0 = <&eth1_rgmii_pins_a &eth1_mdio_pins_a>;
    pinctrl-1 = <&eth1_rgmii_sleep_pins_a &eth1_mdio_sleep_pins_a>;
    pinctrl-names = "default", "sleep";
    phy-mode = "rgmii-id";
    max-speed = <1000>;
    phy-handle = <&phy1_eth1>;
    st,eth-clk-sel;

    mdio1 {
        #address-cells = <1>;
```



```
#size-cells = <0>;
compatible = "snps,dwmac-mdio";

phy1_eth1: ethernet-phy@4 {
    compatible = "ethernet-phy-id4f51.e91b";
    reset-gpios = <&gpiof 3 GPIO_ACTIVE_LOW>;
    reset-assert-us = <10000>;
    reset-deassert-us = <80000>;
    reg = <4>;
};

};

};
```

(2) 屏蔽 eth2 设备树节点

[arch/arm64/boot/dts/st/stm32mp257d-atk-ddr-2GB.dts](#)

```
&eth2 {
    status = "disabled";
    pinctrl-0 = <&eth2_rgmii_pins_a>;
    pinctrl-1 = <&eth2_rgmii_sleep_pins_a>;
    pinctrl-names = "default", "sleep";
    phy-mode = "rgmii-id";
    max-speed = <1000>;
    phy-handle = <&phy1_eth2>;
    st,eth-ptp-from-rcc;

    mdio1 {
        #address-cells = <1>;
        #size-cells = <0>;
        compatible = "snps,dwmac-mdio";
        phy1_eth2: ethernet-phy@1 {
            compatible = "ethernet-phy-id4f51.e91b";
            reset-gpios = <&gpiof 4 GPIO_ACTIVE_LOW>;
            reset-assert-us = <10000>;
            reset-deassert-us = <80000>;
            reg = <1>;
        };
    };
};

};
```

(3) 屏蔽 switch0 设备节点

[arch/arm64/boot/dts/st/stm32mp257d-atk-ddr-2GB.dts](#)

```
&switch0 {
    status = "disabled";
    pinctrl-0 = <&eth1_rgmii_pins_a>, <&eth3_rgmii_pins_a>;
```

```
pinctrl-1 = <&eth1_rgmii_sleep_pins_a>, <&eth3_rgmii_sleep_pins_a>;
pinctrl-names = "default", "sleep";
phy-mode = "rgmii";
st,ethsw-internal-125;
};
```

(4) 系统终端 ENET1 测试验证

启动系统后，若没有看到 end1 设备，则可以手动开启：

ifconfig end1 up

```
root@ATK-DLMP257:~# ifconfig end1 up
[ 58.832182] stm32-dwmac 482c0000.eth1 end1: Register MEM_TYPE_PAGE_POOL RxQ-0
[ 58.834275] stm32-dwmac 482c0000.eth1 end1: Register MEM_TYPE_PAGE_POOL RxQ-1
[ 59.119322] stm32-dwmac 482c0000.eth1 end1: PHY [stmmac-1:04] driver [YT8531 Gigabit Ethernet] (irq=POLL)
[ 59.133524] dwmac4: Master AXI performs any burst length
[ 59.133580] stm32-dwmac 482c0000.eth1 end1: No Safety Features support found
[ 59.342186] stm32-dwmac 482c0000.eth1 end1: IEEE 1588-2008 Advanced Timestamp supported
[ 59.344910] stm32-dwmac 482c0000.eth1 end1: registered PTP clock
[ 59.351478] stm32-dwmac 482c0000.eth1 end1: FPE workqueue start
[ 59.361066] stm32-dwmac 482c0000.eth1 end1: configuring for phy/rgmii-id link mode
root@ATK-DLMP257:~# ifconfig
end1      Link encap:Ethernet  HWaddr 00:11:22:33:44:66
          UP BROADCAST MULTICAST  MTU:1500  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)
          Interrupt:66 Base address:0x8000

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:92 errors:0 dropped:0 overruns:0 frame:0
          TX packets:92 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:7099 (6.9 KiB)  TX bytes:7099 (6.9 KiB)

root@ATK-DLMP257:~# [ 63.455000] stm32-dwmac 482c0000.eth1 end1: Link is Up - 1Gbps/Full - flow control rx/tx
```

图 6.1.2 手动启动 end1 设备

end1 动态获取 IP:

udhcpc -i end1

```
root@ATK-DLMP257:~# udhcpc -i end1
udhcpc: started, v1.36.1
Dropped protocol specifier '.udhcpc' from 'end1.udhcpc'. Using 'end1' (ifindex=2).
udhcpc: broadcasting discover
udhcpc: broadcasting select for 192.168.6.97, server 192.168.6.1
udhcpc: lease of 192.168.6.97 obtained from 192.168.6.1, lease time 86400
/etc/udhcpc.d/50default: Adding DNS 192.168.6.1
Dropped protocol specifier '.udhcpc' from 'end1.udhcpc'. Using 'end1' (ifindex=2).
```

图 6.1.3 动态获取 end1 的 IP 地址

对 end1 设备进行 PING 测试:

ping www.baidu.com -I end1

```
root@ATK-DLMP257:~# ping www.baidu.com -I end1
PING www.baidu.com (183.2.172.177) from 192.168.6.97 end1: 56(84) bytes of data.
64 bytes from 183.2.172.177: icmp_seq=1 ttl=52 time=3.90 ms
64 bytes from 183.2.172.177: icmp_seq=2 ttl=52 time=3.86 ms
64 bytes from 183.2.172.177: icmp_seq=3 ttl=52 time=3.67 ms
64 bytes from 183.2.172.177: icmp_seq=4 ttl=52 time=3.04 ms
64 bytes from 183.2.172.177: icmp_seq=5 ttl=52 time=3.75 ms
^C
--- www.baidu.com ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4006ms
rtt min/avg/max/mdev = 3.043/3.644/3.898/0.310 ms
```

图 6.1.4 对 end1 进行 ping 指令测试

6.2 单独使用 ETH2 (网口 ENET2)

1、修改 UBoot 设备树

以下内容以修改 STM32MP257 DDR 2GB 核心板版本的设备树文件 arch/arm/dts/stm32mp257d-atk-ddr-2GB.dts 为例进行说明。如果你使用的是 1GB DDR 核心板，则应对应修改 arch/arm/dts/stm32mp257d-atk-ddr-1GB.dts 文件。

由于两者在本次需要调整的设备树内容上完全一致，因此此处仅以 2GB 版本为代表进行演示修改，其他版本可参照相同步骤操作即可。

(1) 打开 eth2 设备树节点

arch/arm/dts/stm32mp257d-atk-ddr-2GB.dts

```
&eth2 {
    status = "okay";
    pinctrl-0 = <&eth2_rgmii_pins_a>;
    pinctrl-1 = <&eth2_rgmii_sleep_pins_a>;
    pinctrl-names = "default", "sleep";
    phy-mode = "rgmii-id";
    max-speed = <1000>;
    phy-handle = <&phy1_eth2>;
    st,eth-ptp-from-rcc;

    mdio1 {
        #address-cells = <1>;
        #size-cells = <0>;
        compatible = "snps,dwmac-mdio";
        phy1_eth2: ethernet-phy@1 {
            compatible = "ethernet-phy-id4f51.e91b";
            reset-gpios = <&gpiof 4 GPIO_ACTIVE_LOW>;
            reset-assert-us = <10000>;
            reset-deassert-us = <80000>;
            reg = <1>;
        };
    };
};
```

(2) 屏蔽 eth1 设备树节点

arch/arm/dts/stm32mp257d-atk-ddr-2GB.dts

```
&eth1 {
    status = "disabled";
    pinctrl-0 = <&eth1_mdio_pins_a>;
    pinctrl-names = "default";
    phy-mode = "rgmii";
    st,eth-clk-sel;
    snps,ext-systime;
    fixed_link: fixed-link {
```



```

        speed = <1000>;
        full-duplex;
    };

    mdio1 {
        #address-cells = <1>;
        #size-cells = <0>;
        compatible = "snps,dwmac-mdio";
        phy1_eth1: ethernet-phy@4 {
            compatible = "ethernet-phy-id4f51.e91b",
                "ethernet-phy-ieee802.3-c22";
            reg = <4>;
        };
        phy2_eth1: ethernet-phy@5 {
            compatible = "ethernet-phy-id4f51.e91b",
                "ethernet-phy-ieee802.3-c22";
            reset-gpios = <&gpiof 3 GPIO_ACTIVE_LOW>;
            reset-assert-us = <10000>;
            reset-deassert-us = <80000>;
            reg = <5>;
        };
    };
};

```

(3) 屏蔽 switch0 设备节点

[arch/arm/dts/stm32mp257d-atk-ddr-2GB.dts](#)

```

&switch0 {
    status = "disabled";
    pinctrl-0 = <&eth1_rgmii_pins_a>, <&eth3_rgmii_pins_a>;
    pinctrl-1 = <&eth1_rgmii_sleep_pins_a>, <&eth3_rgmii_sleep_pins_a>;
    pinctrl-names = "default", "sleep";
    phy-mode = "rgmii";
    st,ethsw-internal-125;
};

```

(4) uboot 终端 ENET1 测试验证

首先将网线插入开发板的 ENET1 网口，启动开发板进入 Uboot 终端，输入以下指令初始化 eth1 端口：

```

setenv ethact eth2@482d0000
setenv ipaddr 192.168.6.120
setenv eth1addr 00:11:22:33:44:66
setenv gatewayip 192.168.6.1
setenv netmask 255.255.255.0
setenv serverip 192.168.6.140

```

这里的 **ipaddr**、**gatewayip**、**serverip** 地址应根据自己实际使用情况来修改。

要测试 ENET1 网口是否工作正常，我们可以直接在 U-Boot 环境中使用 ping 指令进行验证。不过要注意，不能随便 ping 任意 IP 地址！只能 ping 前面通过 setenv serverip 设置好的服务器 IP 地址。

```
STM32MP> ping 192.168.6.140
eth2@482d0000 Waiting for PHY auto negotiation to complete..... done
Using eth2@482d0000 device
host 192.168.6.140 is alive
```

图 6.2.1 Uboot 终端测试 ENET1

2、修改 Kernel 设备树

以下内容以修改 STM32MP257 DDR 2GB 核心板版本的设备树文件 arch/arm64/boot/dts/st/stm32mp257d-atk-ddr-2GB.dts 为例进行说明。如果你使用的是 1GB DDR 核心板，则应对应修改 arch/arm64/boot/dts/st/stm32mp257d-atk-ddr-1GB.dts 文件。

由于两者在本次需要调整的设备树内容上完全一致，因此此处仅以 2GB 版本为代表进行演示修改，其他版本可参照相同步骤操作即可。

(1) 屏蔽 eth1 设备树节点

arch/arm64/boot/dts/st/stm32mp257d-atk-ddr-2GB.dts

```
&eth1 {
    status = "disabled";
    pinctrl-0 = <&eth1_mdio_pins_a>;
    pinctrl-names = "default";
    phy-mode = "rgmii";
    st,eth-clk-sel;
    snps,ext-sysstime;
    fixed_link: fixed-link {
        speed = <1000>;
        full-duplex;
    };

    mdio1 {
        #address-cells = <1>;
        #size-cells = <0>;
        compatible = "snps,dwmac-mdio";
        phy1_eth1: ethernet-phy@4 {
            compatible = "ethernet-phy-id4f51.e91b",
                "ethernet-phy-ieee802.3-c22";
            reg = <4>;
        };
        phy2_eth1: ethernet-phy@5 {
            compatible = "ethernet-phy-id4f51.e91b",
                "ethernet-phy-ieee802.3-c22";
            reset-gpios = <&gpiof 3 GPIO_ACTIVE_LOW>;
            reset-assert-us = <10000>;
            reset-deassert-us = <80000>;
            reg = <5>;
        };
    };
};
```



```
};
};
};
```

(2) 打开 eth2 设备树节点

[arch/arm64/boot/dts/st/stm32mp257d-atk-ddr-2GB.dts](#)

```
&eth2 {
    status = "okay";
    pinctrl-0 = <&eth2_rgmii_pins_a>;
    pinctrl-1 = <&eth2_rgmii_sleep_pins_a>;
    pinctrl-names = "default", "sleep";
    phy-mode = "rgmii-id";
    max-speed = <1000>;
    phy-handle = <&phy1_eth2>;
    st,eth-ptp-from-rcc;

    mdio1 {
        #address-cells = <1>;
        #size-cells = <0>;
        compatible = "snps,dwmac-mdio";
        phy1_eth2: ethernet-phy@1 {
            compatible = "ethernet-phy-id4f51.e91b";
            reset-gpios = <&gpiof 4 GPIO_ACTIVE_LOW>;
            reset-assert-us = <10000>;
            reset-deassert-us = <80000>;
            reg = <1>;
        };
    };
};
```

(3) 屏蔽 switch0 设备节点

[arch/arm64/boot/dts/st/stm32mp257d-atk-ddr-2GB.dts](#)

```
&switch0 {
    status = "disabled";
    pinctrl-0 = <&eth1_rgmii_pins_a>, <&eth3_rgmii_pins_a>;
    pinctrl-1 = <&eth1_rgmii_sleep_pins_a>, <&eth3_rgmii_sleep_pins_a>;
    pinctrl-names = "default", "sleep";
    phy-mode = "rgmii";
    st,ethsw-internal-125;
};
```

(4) 系统终端 ENET1 测试验证

启动系统后，若没有看到 end1 设备，则可以手动开启：

```
ifconfig end0 up
```

```
root@ATK-DLMP257:~# ifconfig end0 up
[ 47.094865] stm32-dwmac 482d0000.eth2 end0: Register MEM_TYPE_PAGE_POOL RxQ-0
[ 47.096915] stm32-dwmac 482d0000.eth2 end0: Register MEM_TYPE_PAGE_POOL RxQ-1
[ 47.373106] stm32-dwmac 482d0000.eth2 end0: PHY [stmmac-0:01] driver [YT8531 Gigabit Ethernet] (irq=POLL)
[ 47.385645] dwmac4: Master AXI performs any burst length
[ 47.385699] stm32-dwmac 482d0000.eth2 end0: No Safety Features support found
[ 47.393617] stm32-dwmac 482d0000.eth2 end0: IEEE 1588-2008 Advanced Timestamp supported
[ 47.400854] stm32-dwmac 482d0000.eth2 end0: registered PTP clock
[ 47.406857] stm32-dwmac 482d0000.eth2 end0: FPE workqueue start
[ 47.412628] stm32-dwmac 482d0000.eth2 end0: configuring for phy/rgmii-id link mode
root@ATK-DLMP257:~# ifconfigl 50.491229] stm32-dwmac 482d0000.eth2 end0: Link is Up - 1Gbps/Full - flow control rx/tx

end0      Link encap:Ethernet  HWaddr 00:11:22:33:44:55
          inet addr:192.168.6.66  Bcast:192.168.6.255  Mask:255.255.255.0
          inet6 addr: fe80::211:22ff:fe33:4455/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:429  errors:0  dropped:14  overruns:0  frame:0
          TX packets:55  errors:0  dropped:0  overruns:0  carrier:0
          collisions:0  txqueuelen:1000
          RX bytes:34921 (34.1 KiB)  TX bytes:9053 (8.8 KiB)
          Interrupt:66 Base address:0x8000

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:92  errors:0  dropped:0  overruns:0  frame:0
          TX packets:92  errors:0  dropped:0  overruns:0  carrier:0
          collisions:0  txqueuelen:1000
          RX bytes:7099 (6.9 KiB)  TX bytes:7099 (6.9 KiB)
```

图 6.2.2 手动启动 end1 设备

end0 动态获取 IP:

```
udhcpc -i end0
```

```
root@ATK-DLMP257:~# udhcpc -i end0
udhcpc: started, v1.36.1
Dropped protocol specifier '.udhcpc' from 'end0.udhcpc'. Using 'end0' (ifindex=2).
udhcpc: broadcasting discover
udhcpc: broadcasting select for 192.168.6.66, server 192.168.6.1
udhcpc: lease of 192.168.6.66 obtained from 192.168.6.1, lease time 86400
/etc/udhcpc.d/50default: Adding DNS 192.168.6.1
Dropped protocol specifier '.udhcpc' from 'end0.udhcpc'. Using 'end0' (ifindex=2).
```

图 6.2.3 动态获取 end1 的 IP 地址

对 end1 设备进行 PING 测试:

```
ping www.baidu.com -I end0
```

```
root@ATK-DLMP257:~# ping www.baidu.com -I end0
PING www.a.shifen.com (183.2.172.17) from 192.168.6.66 end0: 56(84) bytes of data.
64 bytes from 183.2.172.17: icmp_seq=1 ttl=52 time=6.84 ms
64 bytes from 183.2.172.17: icmp_seq=2 ttl=52 time=7.74 ms
64 bytes from 183.2.172.17: icmp_seq=3 ttl=52 time=7.03 ms
64 bytes from 183.2.172.17: icmp_seq=4 ttl=52 time=7.06 ms
64 bytes from 183.2.172.17: icmp_seq=5 ttl=52 time=6.75 ms
64 bytes from 183.2.172.17: icmp_seq=6 ttl=52 time=7.26 ms
^C
--- www.a.shifen.com ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5009ms
rtt min/avg/max/mdev = 6.747/7.112/7.743/0.326 ms
```

图 6.2.4 对 end1 进行 ping 指令测试